

“Visitor Approval Entry System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

D . SHAMIKA HASINI 2520080033

M . PRIYAVALLIKA 2520090183

Under the Esteemed Guidance of

Divya Priya Degala

Assistant Professor

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Visitor Approval Entry System”** is a record of bonafide work of **D.SHAMIKA HASINI – 2520080033, M.PRIYAVALLIKA - 2520090183** submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 10, Team 1, during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

D.SHAMIKA HASINI– 2520080033

M.PRIYAVALLIKA – 2520090183

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled **“Event and Room Occupancy Management System”** is a bonafide work done and submitted by **D.SHAMIKA HASINI – 2520080033, M.PRIYAVALLIKA - 2520090183** in partial fulfillment of the requirements of the course **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)**, Section 10, Team 1, at KLH University during **2026–2027, Trimester 4.**

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr** , and my guide **Divya Priya Degala**, Department of Computer Science and Information Technology which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Kaja Shareef**, Head of the *Department for Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

D.SHAMIKA HASINI– 2520080033

M.PRIYAVALLIKA – 2520090183

Abstract

The **Visitor Approval Entry System (VAES)** is a web-based application designed to provide a secure, organized, and efficient solution for managing visitors in residential communities. In traditional visitor management, security personnel commonly depend on manual registers, phone calls, or verbal communication with residents to verify visitors. Such methods can result in delays, difficulty in maintaining accurate visitor records, and problems in managing visitors when residents are unavailable. VAES addresses these challenges by providing a centralized digital platform for managing visitor requests, approvals, pre-approved visits, scheduled visitors, and entry records.

The system provides separate functionalities for **residents/tenants and security personnel**. Security personnel can register visitor details and generate approval requests for the concerned resident. Residents can view the visitor information and approve or reject the request based on their decision. The system also provides **pre-approval and visitor scheduling facilities**, allowing residents to authorize expected visitors in advance. An **absence password** feature is provided for situations where residents are unavailable, while visitor and visit records help maintain organized information about previous entries. Request statuses such as pending, approved, and rejected provide clear information about the current state of each visitor request.

The application is developed using **React.js with Vite** for the frontend, **Node.js and Express.js** for the backend, and **MySQL** for storing and managing application data. Authentication and role-based authorization are implemented to ensure that residents and security personnel can access only the functions relevant to their roles. Password protection and validation mechanisms are also incorporated to improve the security of user accounts and system operations.

Overall, the **Visitor Approval Entry System** aims to reduce manual effort, improve communication between residents and security personnel, and provide a reliable digital record of visitor activities. By centralizing the visitor approval process, the system gives residents greater control over access to their premises while helping security personnel verify and manage visitors more efficiently. The application therefore provides a practical and user-friendly approach to improving visitor management and security within residential communities.

Index

S.No.	Chapters	Topics	Page No.
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	5
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	7
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 Database 3.4 Version Control 3.5 Authentication and Security Libraries	8
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer 4.3 Deployment Architecture	9
5	Design	5.1 Database Design Principles 5.2 ER Diagram / Database Schema 5.3 Design Decisions and Assumptions	10
6	Implementation	6.1 Module-wise Implementation 6.2 Front-end Logic 6.3 API Integration 6.4 Authentication and Authorization	12
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	13
8	Testing	8.1 Postman API Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results	15
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	17
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	18

S.No.	Chapters	Topics	Page No.
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	19
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	20
13	References	– Documentation, tools, and resources used	21
14	Appendices	– Sample Code Snippets – Installation / Setup Instructions – Default Login Credentials – API Reference Summary – Screenshots (to be added by the student)	22

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Residential communities receive visitors such as family members, friends, delivery personnel, and service providers on a regular basis. In traditional visitor management, security personnel often use manual registers, phone calls, or direct communication with residents to verify visitors. These methods can cause delays and make it difficult to maintain accurate visitor records and approval information.

The **Visitor Approval Entry System (VAES)** was developed to provide a centralized digital solution for managing visitor entry. Security personnel can enter visitor details and send approval requests to residents, who can then approve or reject the requests. The system also supports **visitor pre-approval, scheduled visits, absence passwords, and visit records**, helping residents manage visitors more efficiently.

VAES is developed using **React.js and Vite** for the frontend, **Node.js and Express.js** for the backend, and **MySQL** for database management. Authentication and role-based access control provide separate functionalities for residents and security personnel. The system aims to reduce manual work, improve communication, maintain organized visitor records, and provide a more efficient and secure visitor management process.

1.2 Problem Statement

Managing visitors in residential communities through manual registers, phone calls, or verbal communication can cause delays in verification and approval, while maintaining accurate visitor records can be difficult. The **Visitor Approval Entry System (VAES)** provides a centralized platform where security personnel can submit visitor details and residents can approve or reject entry requests. The system also supports pre-approved and scheduled visitors, helping improve communication, reduce manual effort, and provide a more organized and secure visitor management process.

1.3 Scope of the Project

The **Visitor Approval Entry System (VAES)** covers the complete process of managing visitors in a residential community. The system allows security personnel to register visitor details and send approval requests to residents, while residents can approve or reject requests, create pre-approvals, schedule visitors, and manage absence passwords. The system also maintains visitor and visit records with clear approval statuses. Role-based access ensures that residents and security personnel can use the functions relevant to their roles. The project focuses on improving visitor verification, reducing manual entry procedures, maintaining organized records, and providing a centralized platform for efficient and secure visitor management.

1.4 Objectives

- To provide a **secure and centralized platform** for managing visitors in residential communities.
- To allow security personnel to **register visitor details and send approval requests** to residents.
- To enable residents to **approve or reject visitor requests** efficiently.
- To support **visitor pre-approval and scheduled visits** for better planning and convenience.
- To provide an **absence password** facility when residents are unavailable.
- To maintain **organized visitor and visit records** for future reference.
- To implement **role-based authentication and authorization** for residents and security personnel.

1.5 Importance of the Application

The **Visitor Approval Entry System (VAES)** is important because it provides a centralized and organized way to manage visitors in residential communities. It reduces the dependency on manual registers and phone calls by allowing security personnel and residents to communicate through a digital platform. Residents can verify and approve visitors, while security personnel can quickly check the approval status before allowing entry. The system also maintains visitor records and supports pre-approved and scheduled visits, improving convenience, communication, and overall security within the community.

1.6 Target Users / Audience

The **Visitor Approval Entry System (VAES)** is mainly designed for residential communities where visitor verification and controlled entry are required. The primary target users are **residents/tenants** and **security personnel**. Residents use the system to manage visitor requests, approve or reject visitors, create pre-approvals, and schedule visits. Security personnel use the system to register visitor details, send approval requests, and verify the approval status before allowing entry. The system is intended to improve communication between residents and security personnel while providing a centralized and organized visitor management process.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The **Visitor Approval Entry System (VAES)** has moderate hardware requirements since the application uses a React frontend, Node.js backend, and MySQL database. A computer with a **dual-core processor, 4 GB of RAM, and at least 500 MB of available storage** is sufficient for development and demonstration. A stable internet connection is recommended for installing dependencies and accessing the web application. The system can run the frontend, backend, and database on the same computer during development.

2.2 Software Requirements

- Node.js 18 or later, with npm
- MySQL 8.0 or later
- A modern web browser (Chrome, Edge, or Firefox)
- Git, for version control
- MySQL Workbench and Visual Studio Code, for development (optional but recommended)
- github

2.3 Development Tools and Frameworks

React.js with Vite is used to develop the frontend of the application, providing an interactive interface for residents and security personnel. **Node.js with Express.js** is used to develop the backend and handle API requests and application logic. **MySQL** is used as the relational database for storing user, visitor, approval, and visit-related information. **JWT and bcrypt** are used for authentication and secure password handling. **Visual Studio Code** is used as the primary development environment, while **MySQL Workbench** is used to manage and verify the database.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The frontend of the **Visitor Approval Entry System (VAES)** is developed using **React.js** and **Vite**, providing an interactive and responsive web interface for residents and security personnel. The application provides separate interfaces based on the user's role, allowing residents to manage visitor requests and security personnel to register visitors and check approval status. React components are used to organize the interface into reusable sections, while CSS is used for styling and maintaining a consistent user experience across the application.

3.2 Backend

The backend is developed using **Node.js** and **Express.js** and provides the server-side logic required for the visitor management system. It handles user authentication, visitor requests, approvals, pre-approvals, scheduled visits, absence passwords, and visit-related operations. The Express.js server provides APIs through which the frontend communicates with the database and applies the required validation and authorization rules before processing requests.

3.3 Database

MySQL is used as the relational database for storing and managing the application's persistent information. The database maintains details related to users, visitors, approval requests, pre-approvals, schedules, absence passwords, and visit records. Relationships between the stored information help maintain data consistency and allow the system to retrieve relevant visitor and approval details when required.

3.4 Version Control

Git is used for version control during the development of the Visitor Approval Entry System. It helps maintain different versions of the source code and allows changes made during development to be tracked. **GitHub** can be used to store and manage the project repository, making it easier to maintain the frontend and backend source code and collaborate during development.

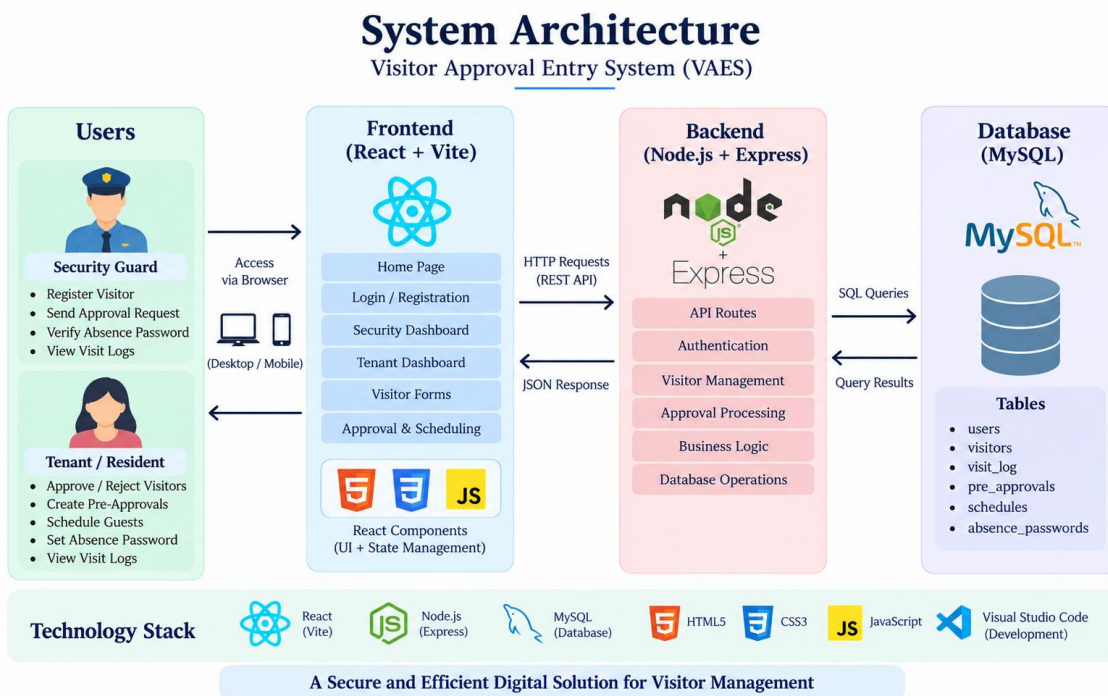
3.5 Authentication and Security Libraries

The system uses **JSON Web Tokens (JWT)** for user authentication and **bcrypt** for secure password hashing. JWT-based authentication helps maintain authenticated user sessions and allows the backend to identify the user's role when accessing protected resources. Role-based authorization ensures that residents and security personnel can access only the functionalities associated with their respective roles. Passwords are stored in hashed form rather than as plain text, providing an additional layer of protection for user accounts.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The system follows a standard three-tier architecture, cleanly separating the user interface, the business logic, and the persisted data. Figure 4.1 shows the three layers and how a request flows between them.



4.2 Description of Each Layer

The **Visitor Approval Entry System (VAES)** follows a layered architecture in which each layer performs a specific function and communicates with the other layers to manage visitor information and approval processes.

The **Presentation Layer** is developed using React.js and Vite and provides the user interface through which residents and security personnel interact with the system. It includes the home page, login and registration, security dashboard, tenant dashboard, visitor forms, and approval and scheduling features.

The **Application Layer** is developed using Node.js and Express.js and provides the REST API for communication between the frontend and database. It handles authentication, visitor management, approval processing, business logic, and database operations. The backend receives HTTP requests from the frontend, processes the required operations, and returns the results in JSON format.

The **Data Layer** uses MySQL to store and manage the system's information, including users, visitors, visit logs, pre-approvals, schedules, and absence passwords. The backend uses SQL queries to insert, retrieve, update, and manage the required records. The **Users Layer** represents the main users of the system, namely security personnel and tenants/residents. Security personnel can register visitors, send approval requests, verify absence passwords, and view visit logs, while residents can approve or reject visitors, create pre-approvals, schedule guests, set absence passwords, and view visit logs.

4.3 Deployment Architecture

The **Visitor Approval Entry System (VAES)** is deployed as a full-stack web application consisting of the frontend, backend, and database components. The **React.js and Vite frontend** runs in the user's web browser and provides the interface for residents and security personnel. The **Node.js and Express.js backend** runs as a server and handles API requests, authentication, visitor approval processing, and application logic. The backend communicates with the **MySQL database**, which stores user, visitor, approval, scheduling, and visit-related information. During development and demonstration, these components can run on the same computer, with the frontend communicating with the backend through REST APIs and the backend communicating with MySQL through database queries.

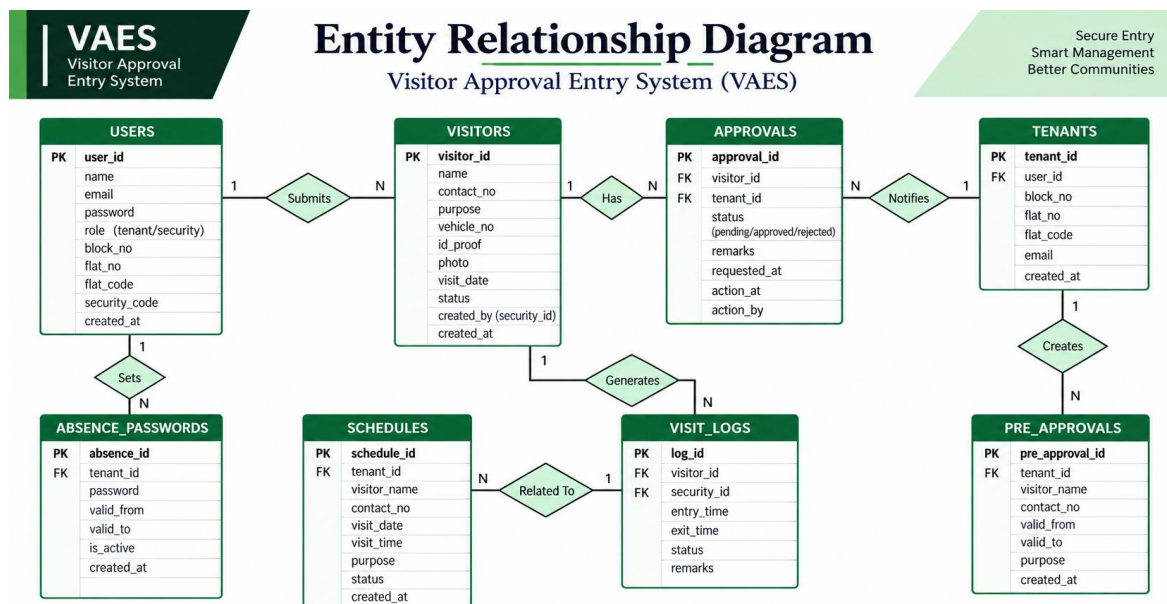
CHAPTER 5: DESIGN

5.1 Database Design Principles

The **Visitor Approval Entry System (VAES)** uses a relational database design in **MySQL** to store and manage user, visitor, approval, scheduling, and visit information. The database is divided into separate tables such as **users**, **visitors**, **visit_log**, **pre_approvals**, **schedules**, and **absence_passwords**, with each table storing information related to a specific part of the system. Primary keys are used to uniquely identify records, while unique constraints help prevent duplicate user information such as email addresses. Foreign key relationships are used where required to maintain data consistency between related records. Passwords are stored in hashed form using **bcrypt** rather than plain text. The design also maintains a separate visit log so that historical visitor information can be retained even when active visitor or schedule records are removed. These principles help maintain organized, consistent, and secure data throughout the application.

5.2 ER Diagram / Database Schema

The database consists of seven related tables, shown in Figure 5.1.



The database consists of **eight related tables**, shown in Figure 5.1. **USERS** stores account details and role information for residents and security personnel. **VISITORS** stores visitor details such as name, contact number, purpose, vehicle number, ID proof, visit date, and status. **APPROVALS** records visitor approval requests and their status, while **TENANTS** stores resident and flat-related information. **PRE_APPROVALS** stores visitors approved in advance by tenants, and **SCHEDULES** maintains details of scheduled visits. **ABSENCE_PASSWORDS** stores temporary absence-password information, while **VISIT_LOGS** records visitor entry and exit details. Primary keys and foreign keys are used to maintain relationships between the tables and ensure organized and consistent data management throughout the application.

5.3 Design Decisions and Assumptions

The VAES database is designed with separate tables for users, visitors, approvals, schedules, pre-approvals, absence passwords, and visit logs. The system assumes that **security personnel register visitors and send approval requests**, while residents approve or reject requests and can create pre-approved or scheduled visits. Role-based access is used to provide appropriate functions to each type of user.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The **Visitor Approval Entry System (VAES)** is implemented using separate modules for authentication, visitor management, approvals, pre-approvals, scheduling, absence passwords, and visit logs. The **Authentication module** manages user registration, login, and role-based access for residents and security personnel. The **Visitor Management module** allows security personnel to register visitor details and create approval requests. The **Approval module** enables residents to approve or reject visitor requests, while the **Pre-Approval and Scheduling modules** allow residents to authorize and schedule expected visitors in advance. The **Absence Password module** manages temporary passwords for situations when residents are unavailable, and the **Visit Log module** records visitor entry and exit information for tracking and future reference.

6.2 Front-end Logic

The frontend of the **Visitor Approval Entry System (VAES)** is developed using **React.js** and **Vite**. It provides separate interfaces for residents and security personnel based on their roles. React components are used to manage login, registration, visitor forms, dashboards, approval requests, pre-approvals, scheduling, absence passwords, and visit logs. The frontend communicates with the backend through API requests and updates the displayed information based on the response received from the server. This provides users with an interactive and organized interface for managing visitor-related activities.

6.3 API Integration

The **Visitor Approval Entry System (VAES)** frontend communicates with the Node.js and Express.js backend through **REST APIs**. These APIs are used to handle user authentication, visitor registration, approval requests, pre-approvals, scheduled visits, absence passwords, and visit logs. The frontend sends requests to the appropriate backend endpoints, and the backend processes the request, performs the required database operations, and returns the result to the frontend. This API-based communication provides a clear connection between the user interface, application logic, and MySQL database.

6.4 Authentication and Authorization

The **Visitor Approval Entry System (VAES)** uses authentication and role-based authorization to protect user accounts and system features. Users must log in with their registered credentials before accessing the application. **JWT-based authentication** is used to maintain the user's authenticated session, while passwords are securely stored using **bcrypt hashing**. Role-based authorization separates the permissions of **residents and security personnel**, ensuring that each user can access only the functions provided for their respective role. This helps protect visitor information and prevents unauthorized access to restricted system operations.

CHAPTER 7: FEATURES

7.1 List of Main Features

Resident/Tenant-facing features:

- Register and log in securely to the system.
- View and manage visitor approval requests.
- Approve or reject visitor requests.
- Create pre-approvals for expected visitors.
- Schedule visitors for planned visits.
- Set and manage an absence password.
- View visitor and visit log information.
-

Security Personnel-facing features:

- Register and log in as security personnel.
- Enter and register visitor details.
- Send visitor approval requests to residents.
- Verify visitor approval or absence-password information.
- Track the status of visitor requests.
- View and maintain visit entry and exit records.

7.2 How Each Feature Works

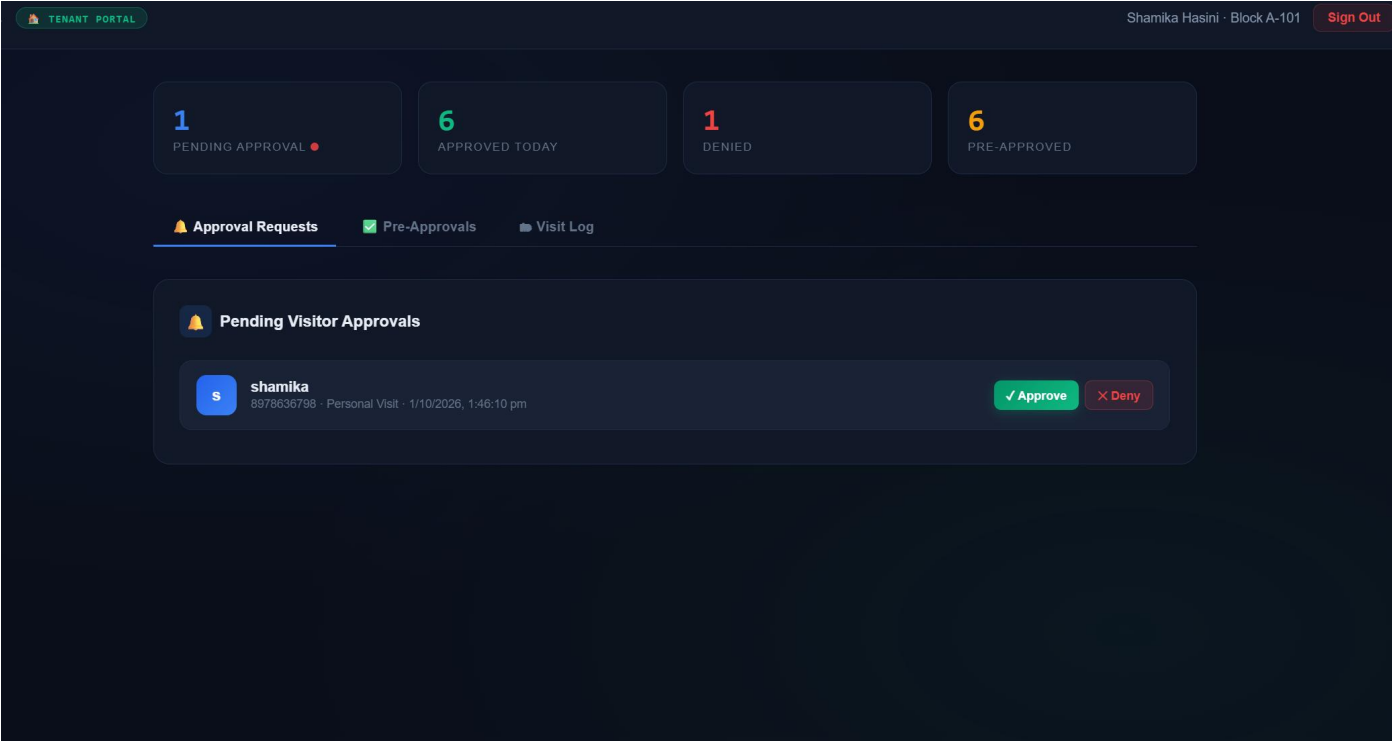
Security personnel can enter the visitor's details through the visitor registration form and submit an approval request to the concerned resident. The resident receives the request on the tenant dashboard and can review the visitor information before approving or rejecting the request. The request status is updated accordingly and can be viewed by the security personnel.

The screenshot displays a web interface for 'New Visitor Registration'. At the top, there are four summary cards: '36 TODAY'S VISITORS', '4 PENDING APPROVAL', '18 APPROVED', and '3 DENIED'. Below these is a navigation bar with three tabs: 'Register Visitor' (active), 'Sent Requests', and 'Visit Log'. The main form area is titled 'New Visitor Registration' and contains the following fields:

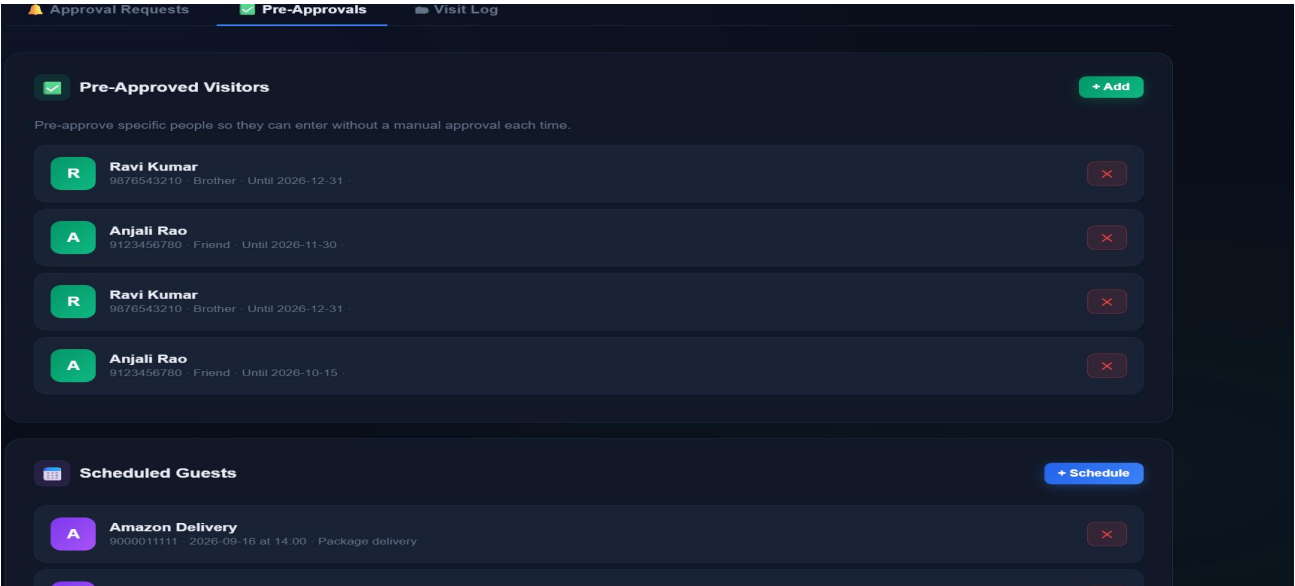
- VISITOR FULL NAME:** A text input field with the placeholder 'Full name of visitor'.
- PHONE NUMBER:** A text input field with the placeholder '+91 00000 00000'.
- PURPOSE OF VISIT:** A dropdown menu with the placeholder 'Select purpose'.
- BLOCK:** A text input field with the placeholder 'e.g. A, B, C'.
- FLAT NUMBER:** A text input field with the placeholder 'e.g. 304'.

At the bottom of the form is a blue button labeled 'Send Approval Request to Tenant'.

When a visitor request is submitted, the resident can view it under the approval requests section. The resident can check the visitor details and select **Approve** or **Reject**. The selected action is stored in the system and the request status changes accordingly. four labels: below 60% is “Normal,” 60–89% is “High,” 90–99% is “Near Capacity,” and 100% or above is “Over Capacity” — shown to the admin as a colored progress bar.



Residents can create a pre-approval for visitors who are expected in advance. The resident enters information such as visitor name, contact number, validity period, and purpose of the visit. The pre-approval is then stored and displayed in the tenant dashboard.



CHAPTER 8: TESTING

8.1 Postman API Testing

The REST APIs of the **Visitor Approval Entry System (VAES)** were tested by running the application and verifying the communication between the frontend and backend. The API operations related to authentication, visitor registration, approval requests, pre-approvals, scheduling, absence-password verification, and visit logs were checked using different inputs. The responses were verified to ensure that valid requests were processed correctly and invalid or unauthorized requests were handled appropriately.

8.2 Integration Testing

The system was tested end-to-end to verify the communication between the **React frontend, Node.js and Express.js backend, and MySQL database**. User actions performed through the frontend were checked to ensure that the corresponding API requests were processed correctly and that the required information was stored or retrieved from the database. The complete visitor workflow, starting from visitor registration and approval request through resident approval or rejection and visit-log management, was also tested to verify that all modules work together correctly.

8.3 Tools Used for Testing

- **Web Browser** — testing the application interface and user workflows
- **Visual Studio Code** — running and debugging the frontend and backend
- **MySQL / MySQL Workbench** — checking database records and data changes
- **Node.js and npm** — running and testing the backend and frontend applications

8.4 Test Cases and Results

A representative sample of the test cases executed, drawn from the project's actual test log, is summarized below.

#	Test Case	Expected Result	Actual Result
1	Login with valid credentials	User successfully logs in	Pass
2	Login with invalid credentials	Login is rejected with an error message	Pass
3	Register a new user	User account is created successfully	Pass
4	Security registers a visitor	Visitor details are stored successfully	Pass
5	Send visitor approval request	Request is sent to the concerned resident	Pass
6	Resident approves visitor	Request status changes to Approved	Pass
7	Resident rejects visitor	Request changes to Rejected	Pass

#	Test Case	Expected Result	Actual Result
8	Create visitor pre-approval	Pre-approval is created successfully	Pass
9	Schedule a visitor	Scheduled visitor details are stored	Pass
10	Set an absence password	Absence password is created successfully	Pass
11	Verify absence password	Valid password is accepted	Pass
12	View visit logs	Previous visitor records are displayed	Pass
13	Unauthorized user accesses restricted feature	Access is denied	Pass

A representative set of test cases was performed to verify the major functionalities and security rules of the **Visitor Approval Entry System**. The test cases covered authentication, visitor registration, approval processing, pre-approval, scheduling, absence-password verification, and visit-log operations. The results were checked against the expected behavior to ensure that the system functions correctly under normal and invalid conditions.

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

The application is deployed locally for development and demonstration as follows:

- Load the schema and seed data: `mysql -u root -p < database/schema.sql`, then ... `< database/seed.sql`
- Start the backend: `cd server, npm install, npm run dev` (listens on port 4000)
- Start the frontend: `cd client, npm install, npm run dev` (listens on port 4173)
- Open `http://localhost:5173` in a browser and log in with a seeded account

9.2 Environment Configuration

The backend configuration is maintained using environment variables to store important application settings such as the **MySQL database connection details, JWT secret, server port, and other required configuration values**. Sensitive information such as database passwords and authentication secrets should not be directly written into the source code. The environment configuration allows the application to connect correctly with the database and run securely during development.

9.3 Hosting of Frontend and Backend

For the current project, the frontend, backend, and MySQL database are run locally for development and demonstration. The React application runs through the Vite development server, while the Node.js and Express.js backend handles API requests and communicates with the MySQL database. A production or cloud deployment is not included in the current implementation, but the application can be adapted for hosted deployment in the future..

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

During the development of the Visitor Approval Entry System (VAES), several challenges were encountered. Managing communication between the React frontend, Node.js and Express.js backend, and MySQL database required careful integration. Implementing separate functionalities for security personnel and residents also required proper role-based access control. Additional challenges included handling visitor approval requests, maintaining visitor and visit records, validating user inputs, and ensuring that the different visitor-management features worked correctly together

10.2 Solutions Applied

The development issues were addressed by separating the application into frontend, backend, and database components with clearly defined responsibilities. Role-based authentication and authorization were implemented to provide appropriate access to residents and security personnel. API-based communication was used to connect the frontend with the backend, while MySQL was used to maintain structured visitor and approval information. The visitor approval, pre-approval, scheduling, absence-password, and visit-log modules were tested individually and together to identify and resolve integration issues.

10.3 Current Limitations or Known Bugs

- The system is currently designed mainly for **local development and demonstration** and has not been deployed to a public production environment.
- The system depends on residents responding to visitor approval requests before normal visitor entry can be completed.
- Visitor information and approval records depend on the data entered by users and security personnel.
- Advanced features such as **facial recognition, QR-code-based entry, and automated visitor identification** are not currently implemented.
- The system can be further enhanced with additional notification and monitoring capabilities in future versions.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

The following features can be added in future versions of the **Visitor Approval Entry System (VAES)**:

- **Email and SMS notifications** for visitor approval requests and important updates.
- **QR-code-based visitor entry** to make visitor verification faster and reduce manual checking.
- **Digital visitor pass generation** for approved visitors.
- **Advanced visitor search and filtering** for quickly finding previous visitor records.
- **Real-time notifications** to inform residents immediately when a visitor request is created.
- **Facial recognition integration** for additional visitor verification and security.
- **Mobile application support** so residents and security personnel can access the system through smartphones.

11.2 Possible Integrations or Optimizations

- Integrating a **notification service** for automated email, SMS, or push notifications.
- Integrating **QR-code scanning** with the security dashboard for faster visitor verification.
- Optimizing database queries and indexing to improve performance when the number of visitor records increases.
- **Containerizing the application with Docker** to provide a consistent deployment environment.
- Deploying the frontend, backend, and database to a **cloud environment** for remote access and centralized management.
- Adding automated testing to reduce errors and ensure that new changes do not affect existing functionality.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

The **Visitor Approval Entry System (VAES)** was developed to provide a centralized and organized solution for managing visitor entry in residential communities. The system provides a digital platform connecting security personnel and residents, allowing visitor registration, approval requests, pre-approvals, scheduled visits, absence-password management, and visit-log maintenance. The application was developed using **React.js and Vite for the frontend, Node.js and Express.js for the backend, and MySQL for database management.**

12.2 What Was Achieved

The completed system provides **role-based authentication** for residents and security personnel and supports the complete visitor approval workflow. Security personnel can register visitors and send approval requests, while residents can approve or reject requests, create pre-approvals, schedule visitors, and manage absence passwords. The system also maintains organized visitor and visit records and provides clear request statuses. These features help reduce manual work, improve communication, and provide a more systematic approach to visitor management.

12.3 Skills Learned

Working on this project provided practical experience in **full-stack web development, React.js, Node.js, Express.js, and MySQL database management.** The project also helped in understanding REST API communication, authentication and role-based authorization, relational database design, frontend and backend integration, and debugging application issues. Developing and testing the different visitor-management modules provided practical experience in building a complete web-based application from design through implementation and testing.

CHAPTER 13: REFERENCES

- React — Official Documentation, <https://react.dev>
- Vite — Official Documentation, <https://vitejs.dev>
- Node.js — Official Documentation, <https://nodejs.org/en/docs>
- Express.js — Official Documentation, <https://expressjs.com>
- MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/refman/8.0/en/>
- mysql2 (Node.js MySQL driver) — npm documentation, <https://www.npmjs.com/package/mysql2>
- jsonwebtoken — npm documentation, <https://www.npmjs.com/package/jsonwebtoken>
- bcrypt — npm documentation, <https://www.npmjs.com/package/bcrypt>
- **MDN Web Docs** — JavaScript and Web APIs Reference, <https://developer.mozilla.org>

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippet

The following code snippet demonstrates the authentication middleware used to protect authorized routes in the **Visitor Approval Entry System (VAES)**:

```
const jwt = require("jsonwebtoken");
const authenticateToken = (req, res, next) => {
  const authHeader = req.headers["authorization"];
  const token = authHeader && authHeader.split(" ")[1];
  if (!token) {
    return res.status(401).json({
      message: "Access token required"
    });
  }
  jwt.verify(token, process.env.JWT_SECRET, (err, user) => { if (err) { return
```

Appendix B: Installation and Setup Instructions

- `mysql -u root -p < database/schema.sql`
- `mysql -u root -p < database/seed.sql`
- `cd server && npm install && npm run dev` (backend, port 4000)
- `cd client && npm install && npm run dev` (frontend, port 4173)
- Open `http://localhost:5173` in a browser

Appendix C: Default Login Credentials

All seeded accounts use the password “shamika200”.

Role	Email
Security Guard	ramya@gmail.com
Shamika Hasini	shamikahasini@gmail.com
Udaya	udaya@gmail.com
Kishore	kishore@gmail.com

Appendix D: API Reference Summary

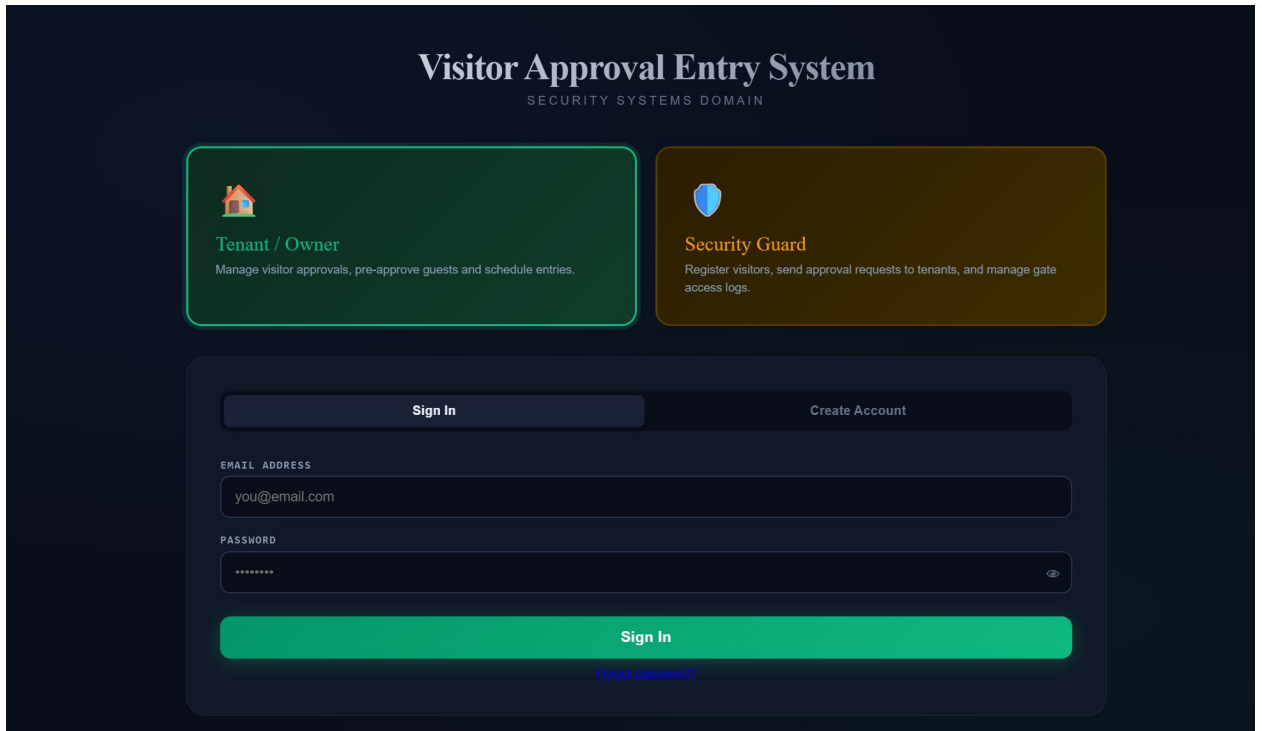
22 endpoints across six resources; full request/response examples are in the accompanying Postman collection.

Resource	Endpoints
Auth	POST /register, POST /login, GET /me
Rooms	GET /, GET /:id, GET /available, POST /, PUT /:id, DELETE /:id
Events	GET /, GET /:id, POST /, PUT /:id, DELETE /:id, PUT /:id/assign-room
Registrations	POST /, GET /my, GET /event/:eventId, GET /
Bookings	GET /, POST /, PUT /:id, DELETE /:id
Occupancy	GET /, POST /, PUT /:id

Appendix E: Screenshots

The following pages are left for the student to complete with actual screenshots taken from the running application before submission. Suggested screenshots, in order:

1. Login Page



The screenshot shows the login page of the "Visitor Approval Entry System" under the "SECURITY SYSTEMS DOMAIN". It features two main role-based buttons: "Tenant / Owner" (green) and "Security Guard" (brown). Below these is a sign-in form with fields for "EMAIL ADDRESS" (containing "you@email.com") and "PASSWORD" (masked with "*****"). A "Sign In" button is at the bottom of the form, with a "Forgot password?" link below it. A "Create Account" button is also present at the top right of the form area.

Visitor Approval Entry System
SECURITY SYSTEMS DOMAIN

Tenant / Owner
Manage visitor approvals, pre-approve guests and schedule entries.

Security Guard
Register visitors, send approval requests to tenants, and manage gate access logs.

Sign In Create Account

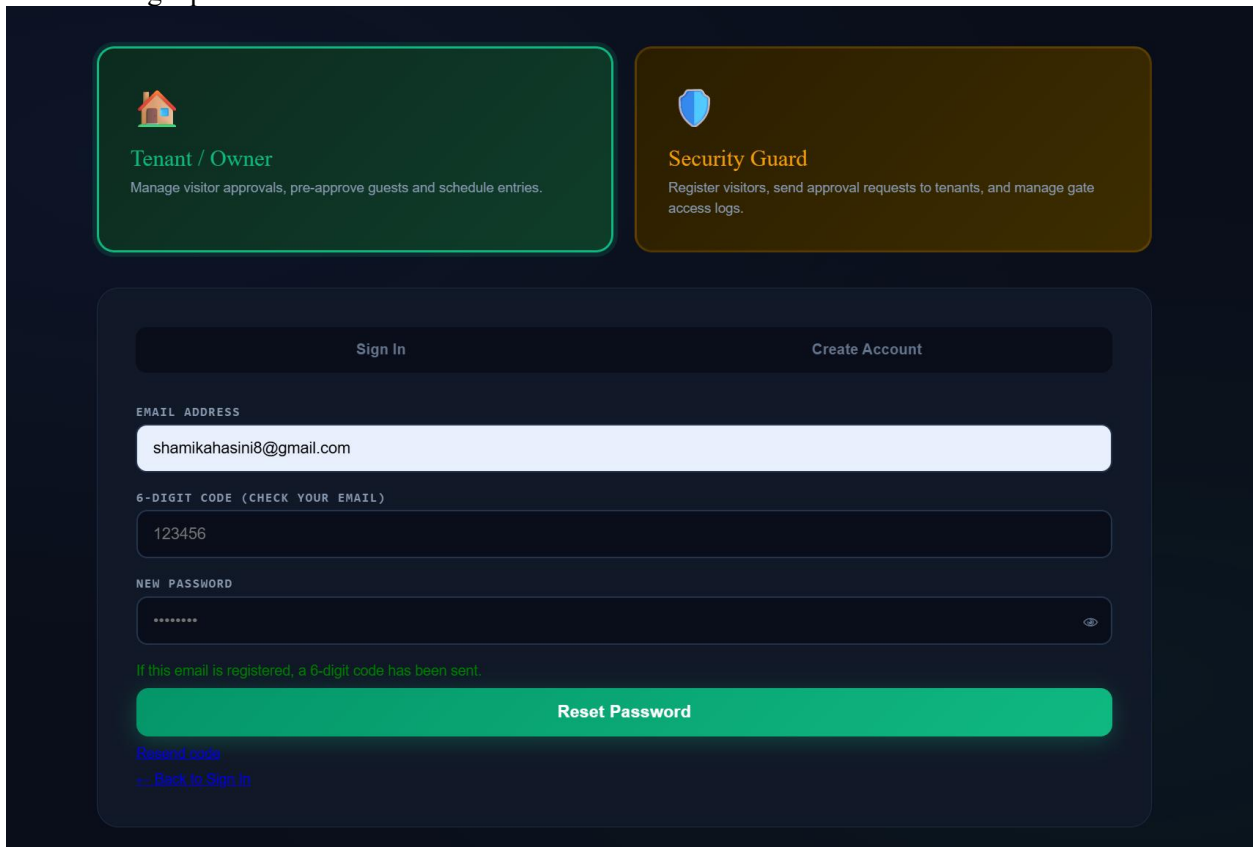
EMAIL ADDRESS
you@email.com

PASSWORD

Sign In

[Forgot password?](#)

2. Forget password



The screenshot shows the "Forget password" page of the "Visitor Approval Entry System". It features the same role-based buttons as the login page. The form includes fields for "EMAIL ADDRESS" (containing "shamikahasini8@gmail.com"), "6-DIGIT CODE (CHECK YOUR EMAIL)" (containing "123456"), and "NEW PASSWORD" (masked with "*****"). A "Reset Password" button is at the bottom of the form. A message states: "If this email is registered, a 6-digit code has been sent." Below the button are links for "Resend code" and "Back to Sign In".

Tenant / Owner
Manage visitor approvals, pre-approve guests and schedule entries.

Security Guard
Register visitors, send approval requests to tenants, and manage gate access logs.

Sign In Create Account

EMAIL ADDRESS
shamikahasini8@gmail.com

6-DIGIT CODE (CHECK YOUR EMAIL)
123456

NEW PASSWORD

If this email is registered, a 6-digit code has been sent.

Reset Password

[Resend code](#)
[Back to Sign In](#)

3. Sign up page



Tenant / Owner

Manage visitor approvals, pre-approve guests and schedule entries.



Security Guard

Register visitors, send approval requests to tenants, and manage gate access logs.

Sign In

Create Account

FULL NAME

Your full name

PHONE

+91 00000 00000

BLOCK

e.g. A, B, C

FLAT NUMBER

e.g. 304

TENANT EMAIL

your@email.com

FLAT CODE (GIVEN BY MANAGEMENT)

e.g. flat-001

This code is provided by your society management office.

SET PASSWORD

Min. 8 characters



4. Tenant Dashboard

VAES

TENANT PORTAL

Shamika Hasini · Block A-101

Sign Out

1

PENDING APPROVAL

6

APPROVED TODAY

1

DENIED

6

PRE-APPROVED

Approval Requests

Pre-Approvals

Visit Log

Pending Visitor Approvals

s

shamika

8978636798 · Personal Visit · 1/10/2026, 1:46:10 pm

✓ Approve

✗ Deny

5. Pre-approvals

✓ Pre-Approved Visitors

+ Add

Pre-approve specific people so they can enter without a manual approval each time.

R

Ravi Kumar

9876543210 · Brother · Until 2026-12-31

✗

A

Anjali Rao

9123456780 · Friend · Until 2026-11-30

✗

R

Ravi Kumar

9876543210 · Brother · Until 2026-12-31

✗

A

Anjali Rao

9123456780 · Friend · Until 2026-10-15

✗

Scheduled Guests

+ Schedule

A

Amazon Delivery

9000011111 · 2026-09-16 at 14:00 · Package delivery

✗

A

Amazon Delivery

9000011111 · 2026-09-15 at 14:00 · Package delivery

✗

6. Security Dashboard and register form

VAES

SECURITY GUARD

Ramya

Sign Out

37

TODAY'S VISITORS

5

PENDING APPROVAL

18

APPROVED

3

DENIED

Register Visitor

Sent Requests

Visit Log

New Visitor Registration

VISITOR FULL NAME

Full name of visitor

PHONE NUMBER

+91 00000 00000

PURPOSE OF VISIT

Select purpose

BLOCK

e.g. A, B, C

FLAT NUMBER

e.g. 304

Send Approval Request to Tenant

Verify Absence Password

7. Absence password

VAES

SECURITY GUARD

Ramya

Sign Out

VISITOR FULL NAME

Full name of visitor

PHONE NUMBER

+91 00000 00000

PURPOSE OF VISIT

Select purpose

BLOCK

e.g. A, B, C

FLAT NUMBER

e.g. 304

Send Approval Request to Tenant

Verify Absence Password

Enter the flat number and the absence password set by the tenant to grant entry.

BLOCK

e.g. A

FLAT NO.

e.g. 101

ABSENCE PASSWORD

Enter tenant's absence password

Verify & Grant Access

8. Visit log with export option

37
TODAY'S VISITORS

5
PENDING APPROVAL

18
APPROVED

3
DENIED

Register Visitor

Sent Requests

Visit Log

Complete Visit Log

Export

VISITOR	BLOCK/FLAT	PURPOSE	TIME	STATUS	ENTRY VIA
SCHED Plumber - Suresh	B-402	Leak repair	2026-09-18 10:30	SCHEDULED	Pre-Scheduled
SCHED Amazon Delivery	A-101	Package delivery	2026-09-15 14:00	SCHEDULED	Pre-Scheduled
SCHED Flipkart Courier	D-204	Package delivery	2026-09-23 13:00	SCHEDULED	Pre-Scheduled
SCHED Swiggy Genie	C-203	Package pickup	2026-09-16 20:00	SCHEDULED	Pre-Scheduled
SCHED Gas Cylinder Delivery	C-201	Cylinder refill	2026-09-22 15:00	SCHEDULED	Pre-Scheduled
SCHED Blue Dart Courier	C-202	Document delivery	2026-09-21 12:00	SCHEDULED	Pre-Scheduled
SCHED Zomato Delivery	B-403	Food delivery	2026-09-16 19:30	SCHEDULED	Pre-Scheduled
SCHED Housekeeping Staff	B-401	Deep cleaning	2026-09-20 08:00	SCHEDULED	Pre-Scheduled